

ui.tooltip 全面详解

ui.tooltip 是 NiceGUI 基于 Quasar 的 QTooltip 组件封装的悬浮提示元素，用于在用户鼠标悬浮于目标元素时显示补充信息，支持纯文本、HTML、图片等多种内容形式，核心用于提升界面交互的友好性（如功能说明、操作提示、数据补充等）。其核心特性包括轻量化集成、灵活的内容支持、样式自定义、触发延迟配置等，是界面中不可或缺的辅助交互组件。

一、核心概念与基础特性

1. 本质与用途

- 本质：依附于目标元素的悬浮提示组件，通过鼠标悬浮触发显示，默认鼠标离开后自动隐藏，无需额外交互逻辑。
- 核心用途：在不占用界面常驻空间的前提下，提供额外信息，常见场景包括：
 - 功能按钮：说明按钮用途（如“删除选中项”“导出数据”）；
 - 表单字段：提示输入格式（如“手机号需为 11 位数字”）；
 - 数据展示：补充数据详情（如表格单元格悬浮显示完整内容）；
 - 复杂元素：解释图标、缩写的含义（如“API 接口文档”）。
- 关键机制：
 - 触发方式：默认鼠标悬浮目标元素触发，无需手动绑定事件；
 - 显示逻辑：悬浮一段时间后显示（可配置延迟），鼠标离开目标元素或提示框后自动关闭；
 - 内容支持：可直接传入文本，或嵌套 HTML、图片、组件等复杂内容；
 - 位置适配：自动贴合目标元素，页面边缘会自动调整位置以确保完全显示。

2. 基础结构

ui.tooltip 有两种核心使用方式：**嵌套式**（支持自定义样式和复杂内容）和 **方法式**（简洁快捷，仅支持纯文本），基础示例如下：

```
from nicegui import ui

# 1. 嵌套式（推荐，支持样式和复杂内容）
with ui.button('悬浮查看提示', classes='mt-4'):
    ui.tooltip('这是基础文本提示').classes('bg-blue text-white') # 自定义背景和文字颜色

# 2. 方法式（简洁，仅支持纯文本，无自定义样式）
ui.label('用方法添加提示').tooltip('方法式提示仅支持纯文本')

# 3. 复杂内容（嵌套图片、HTML）
with ui.icon('info', size='24px', classes='mt-4'):
    with ui.tooltip().classes('bg-transparent p-0'): # 透明背景，无内边距
        ui.image('https://picsum.photos/id/377/200/100').classes('rounded') # 图片提示

with ui.label('HTML 提示', classes='mt-4'):
    with ui.tooltip():
        ui.html('<b>加粗</b>、<em>斜体</em>、<u>下划线</u>', sanitize=False) # HTML 内容
```

`ui.run()`

二、初始化配置项

初始化 `ui.tooltip()` 时可通过参数配置文本、显示行为等核心属性，参数说明如下：

参数名	类型	说明
text	str	提示框默认文本（默认空字符串，若嵌套其他元素，此参数可忽略）
delay	int	悬浮后延迟显示时间（单位：毫秒，默认值由 Quasar 组件决定，可通过 props 配置）
placement	str	提示框显示位置（可选值： <code>top</code> / <code>bottom</code> / <code>left</code> / <code>right</code> / <code>top-start</code> 等，默认自动适配）

配置示例

```
from nicegui import ui

# 配置延迟显示、指定位置的提示框
with ui.button('延迟+指定位置提示', classes='mt-4'):
    ui.tooltip('悬浮1秒后显示，位于按钮下方').props('delay=1000 placement=bottom').classes('bg-green')

# 无默认文本，纯嵌套内容
with ui.input(placeholder='输入手机号', classes='mt-4'):
    with ui.tooltip():
        ui.column(
            ui.label('输入规则：').classes('font-bold'),
            ui.label('1. 仅支持数字'),
            ui.label('2. 长度为11位'),
            classes='p-2 bg-gray-100 rounded'
        )
```

三、核心属性

`ui.tooltip` 继承 NiceGUI 基础元素的通用属性，支持样式配置、状态绑定等，关键属性如下（含可动态修改的属性）：

属性名	类型	说明
classes	str	CSS 类名（支持 Tailwind、Quasar 类，如 <code>bg-red</code> 红色背景、 <code>rounded-lg</code> 圆角、 <code>p-2</code> 内边距）
props	str	Quasar 组件属性（如 <code>delay=500</code> 延迟显示、 <code>transition-show=fade</code> 显示动画、 <code>placement=right</code> 显示位置）

属性名	类型	说明
style	str	内联 CSS 样式 (如 <code>font-size: 14px; padding: 8px;</code> 调整字体大小和内边距)
text	BindableProperty	提示框文本 (支持双向绑定, 动态修改提示内容)
visible	BindableProperty	元素可见性 (布尔值, 支持动态绑定, <code>True</code> 显示、 <code>False</code> 隐藏, 默认自动控制)
html_id	str	HTML DOM 中的元素 ID (版本 2.16.0 新增, 用于精准定位)
is_deleted	bool	元素是否已被删除 (只读属性)
parent_slot	Slot None	父容器的插槽 (可手动设置, 用于复杂布局嵌套)

属性使用示例

```
from nicegui import ui

# 动态修改提示文本和样式
with ui.button('动态提示', classes='mt-4') as btn:
    tooltip = ui.tooltip('初始提示').classes('bg-gray')

# 动态修改文本
def update_text():
    tooltip.set_text('修改后的提示文本')
    ui.notify('提示文本已更新')

# 动态修改样式
def update_style():
    tooltip.classes('bg-purple text-white rounded-lg p-3')
    tooltip.props('delay=800 placement=top')
    ui.notify('提示样式已更新')

ui.row(
    ui.button('更新文本', on_click=update_text),
    ui.button('更新样式', on_click=update_style)
).classes('mt-2')

ui.run()
```

四、核心方法

ui.tooltip 提供丰富的方法用于控制文本、绑定状态、管理元素等，常用方法分类如下：

1. 文本与状态控制方法

方法名	作用	示例
set_text(text: str)	动态修改提示框文本（仅适用于纯文本场景，嵌套复杂元素时无效）	<code>tooltip.set_text('新的提示文本')</code>
set_visibility(visible: bool)	手动设置提示框可见性（默认自动控制，手动设置后会覆盖自动逻辑）	<code>tooltip.set_visibility(True)</code>
show()	手动显示提示框（较少用，默认悬浮触发）	<code>tooltip.show()</code>
hide()	手动隐藏提示框（较少用，默认自动关闭）	<code>tooltip.hide()</code>
update()	强制更新客户端的提示框状态（如动态修改样式或文本后刷新）	<code>tooltip.update()</code>

2. 绑定方法

支持将提示文本、可见性与外部变量绑定，实现动态同步，核心方法如下：

方法名	作用	关键参数
bind_text(target_object, target_name='text')	双向绑定提示文本到目标对象的属性	target_object: 绑定目标对象； target_name: 绑定的属性名（默认 'text'）
bind_text_from(...)	单向绑定（从目标对象同步到提示框）	同 bind_text，仅单向同步（目标对象属性变化触发提示文本更新）
bind_text_to(...)	单向绑定（从提示框同步到目标对象）	同 bind_text，仅单向同步（提示文本变化触发目标对象属性更新，较少用）
bind_visibility(...)	双向绑定可见性到目标对象的属性	value: 可选，指定目标值匹配时才显示提示框

3. 其他常用方法

方法名	作用	示例
clear()	清空提示框内所有嵌套元素（适用于复杂内容场景）	<code>tooltip.clear()</code>
delete()	删除提示框元素	<code>tooltip.delete()</code>
add_resource(path)	为提示框添加资源文件（如自定义 CSS、JS）	<code>tooltip.add_resource('./static')</code>
mark(*markers)	为元素添加标记（用于测试或元素查询）	<code>tooltip.mark('info-tooltip', 'form')</code>

方法使用示例

```
from nicegui import ui

# 绑定提示文本到外部变量
class TooltipState:
    tip_text = '初始绑定文本'
    is_visible = False

state = TooltipState()

with ui.button('绑定状态的提示', classes='mt-4'):
    tooltip = ui.tooltip()
    # 双向绑定文本和可见性
    tooltip.bind_text(state, 'tip_text')
    tooltip.bind_visibility(state, 'is_visible')

# 控制按钮
ui.row(
    ui.button('修改文本', on_click=lambda: setattr(state, 'tip_text', '修改后的绑定文本')),
    ui.button('显示提示', on_click=lambda: setattr(state, 'is_visible', True)),
    ui.button('隐藏提示', on_click=lambda: setattr(state, 'is_visible', False))
).classes('mt-2')

ui.run()
```

五、特殊场景适配

1. 不支持嵌套的元素（如 ui.html、ui.markdown、ui.upload）

部分元素（如 `ui.html`、`ui.markdown`、`ui.upload`）不支持直接嵌套 `ui.tooltip`，需通过 `ui.element()` 容器包裹后添加提示：

```
from nicegui import ui

# 1. 为 ui.html 添加提示
with ui.element().classes('mt-4'):
    ui.html('<u>不支持直接嵌套的 HTML 元素</u>', sanitize=False)
    ui.tooltip('通过容器包裹添加提示')

# 2. 为 ui.upload 添加提示
with ui.element(classes='mt-4'):
    ui.upload(on_upload=lambda e: ui.notify(f'上传文件: {e.file.name}')).classes('w-64')
    ui.tooltip('支持上传图片、文档等文件').props('delay=800')

ui.run()
```

2. 复杂内容提示（组件、列表、交互元素）

提示框内可嵌套任意 NiceGUI 组件，甚至支持简单交互（如按钮），示例如下：

```
from nicegui import ui

with ui.button('复杂内容提示', classes='mt-4'):
    with ui.tooltip().classes('p-3 bg-gray-100 rounded-lg'):
        ui.column(
            ui.label('复杂提示包含: ').classes('font-bold mb-2'),
            ui.row(
                ui.icon('check_circle', color='green'),
                ui.label('列表项1')
            ),
            ui.row(
                ui.icon('check_circle', color='green'),
                ui.label('列表项2')
            ),
            ui.button('提示内按钮', on_click=lambda: ui.notify('点击了提示内的按钮')).classes('mt-2 bg-blue text-white')
        )

ui.run()
```

3. 样式深度自定义（品牌化适配）

结合 Tailwind CSS 和 Quasar props 实现个性化样式，适配产品风格：

```
from nicegui import ui

# 品牌化提示框：圆角、阴影、渐变背景、自定义字体
with ui.button('品牌化提示', classes='mt-4 bg-purple-600 text-white'):
    ui.tooltip('品牌专属悬浮提示')\
        .classes('rounded-xl shadow-lg bg-gradient-to-r from-purple-500 to-pink-500 text-white p-3 font-medium')\
        .props('delay=600 placement=top transition-show=scale transition-hide=fade')

ui.run()
```

六、注意事项

1. 两种使用方式区别：

- 嵌套式（`with ui.tooltip(): ...`）：支持自定义样式（`classes/props/style`）和复杂内容（HTML、图片、组件），推荐优先使用；
- 方法式（`element.tooltip('文本')`）：仅支持纯文本，无法自定义样式，适合简单场景快速集成。

2. HTML 安全：嵌套 `ui.html` 时，`sanitize` 参数默认 `True`（过滤危险标签），若需使用完整 HTML 功能，需设为 `False`，但需注意 XSS 风险。

3. 延迟与动画：可通过 `props` 配置 `delay`（延迟显示）、`transition-show`（显示动画）、`transition-hide`（隐藏动画），支持 Quasar 组件的所有动画类型（如 `fade`、`scale`、`rotate`）。

4. 位置配置: `placement` 参数支持多种位置值 (如 `top`、`bottom-start`、`right-end`) , 若不指定, 组件会自动根据目标元素位置和页面边缘适配。
5. 版本兼容性: `html_id` 属性需 NiceGUI 2.16.0+ 版本支持, `bind_text` 的 `strict` 参数需 3.0.0+ 版本支持, 使用时需确认版本匹配。
6. 性能注意: 避免在高频交互元素 (如表格单元格) 中使用过于复杂的提示内容 (如大图片、大量组件) , 可能影响页面性能。

通过以上配置与方法, `ui.tooltip` 可灵活满足从简单文本提示到复杂交互提示的各类需求, 是提升界面交互友好性的核心辅助组件之一。